SPENDWISE

A Personal Expense Tracking and Budget Management System

Project Report

Submitted by:
Agreeta Gupta

Course:
Problem Solving Using Python

Institution:
VIT Bhopal University

Programming Language:
Python

Academic Year:
2026–27

1. Introduction

Managing money can be difficult for students, especially when they have a limited monthly budget. It is easy to lose track of small daily expenses and savings.

SpendWise is a simple Python-based expense tracking and budget management system. It allows users to record income and expenses, manage a monthly budget, check spending, calculate savings, and generate monthly reports.

The project uses basic Python concepts such as variables, lists, loops, conditions, and functions. The main aim of SpendWise is to make personal expense management simple and organized.

2. Problem Statement

Managing daily expenses can be difficult, especially for students who have a limited monthly budget. When expenses are recorded manually or not recorded at all, it becomes difficult to know how much money has been spent and where it has been spent. It can also be difficult to keep track of different types of expenses such as food, travel, shopping, and other personal needs. This may lead to poor budget management and difficulty in calculating savings.

To solve this problem, **SpendWise** is developed as a simple personal expense tracking and budget management system. It allows users to record their income and expenses, organize transactions into categories, set and monitor a monthly budget, analyze spending, calculate savings, and generate monthly reports. The system provides an easy way for users to keep their financial information organized and understand their spending habits.

3. Objectives

The main objectives of the SpendWise project are:

- To keep a record of daily income and expenses.
- To organize expenses into different categories such as food, travel, and shopping.
- To allow users to set a monthly budget and keep track of their spending.
- To show warnings when the budget is almost used or exceeded.
- To calculate total income, total expenses, and savings.
- To show category-wise spending to help users understand where their money is going.
- To generate a monthly report containing important financial information.
- To provide options to edit, delete, and search transactions.
- To provide a simple and easy-to-use system for managing personal expenses.

4. Target Users

- SpendWise is mainly designed for students and individuals who want to keep track of their personal finances. It can be useful for students who receive a fixed amount of money each month and need to manage their expenses within that amount.
- The system can also be used by individuals who want to record their daily income and expenses, monitor their monthly budget, and understand their spending habits. Since the system has a simple menu-based interface, it is suitable for users with basic computer knowledge.

5. Functional Requirements

- The main functional requirements of SpendWise are:

5.1 Add Income

- The system allows the user to enter income details such as amount, category, description, month/year, and payment method.

5.2 Add Expense

- The user can enter details of expenses made during a particular month. Each expense is stored with its amount, category, description, month/year, and payment method.
- **5.3 View Transactions**
- The system allows the user to view all the income and expense transactions that have been entered.
- **5.4 Budget Management**
- The user can enter a monthly budget and the system calculates the total expenses, remaining budget, and percentage of the budget used for that month.
- **5.5 Budget Alerts**
- The system displays different messages when the budget usage is high, almost completely used, or exceeded.
- **5.6 Spending Analysis**
- The system calculates total expenses and shows how much money has been spent in different categories along with their percentages.
- **5.7 Savings Calculation**
- The system calculates savings by subtracting total expenses from total income. It also calculates the savings percentage.
- **5.8 Monthly Report**
- The user can select a particular month and view its total income, expenses, savings, savings percentage, and category-wise expenses.

6. Non-Functional Requirements

6.1 Usability

The system should be simple and easy to use. The menu options and messages should be clear so that a user with basic computer knowledge can operate the program.

6.2 Reliability

The system should give correct results for calculations such as total income, expenses, savings, and budget usage.

6.3 Performance

The system should process the entered transactions and display the results quickly without unnecessary delay.

6.4 Error Handling

The system should prevent invalid values such as zero or negative amounts for income, expenses, and budgets.

6.5 Maintainability

The program is divided into different functions for different operations. This makes the code easier to understand, modify, and maintain.

6.6 Accuracy

The calculations performed by the system, such as savings, budget percentage, and category-wise expenses, should be accurate according to the data entered by the user.

7. Major Modules

SpendWise is divided into the following major modules:

7.1 Transaction Management Module

This module is used to manage income and expense transactions. It allows the user to add, view, edit, delete, and search transactions. Each transaction is given a unique ID during normal program use.

7.2 Budget Management Module

This module helps the user manage a monthly budget. It calculates the total expenses for a selected month, the remaining budget, and the percentage of the budget that has been used. It also displays warnings when the budget is almost used or exceeded.

7.3 Spending Analysis Module

This module analyzes the user's expenses and displays the total spending for different categories. It also shows the percentage of total expenses spent in each category.

7.4 Savings Module

This module calculates the user's total income, total expenses, savings, and savings percentage.

7.5 Monthly Report Module

This module provides a summary for a selected month. It displays monthly income, expenses, savings, savings percentage, and category-wise expenses.

8. System Workflow

The workflow of SpendWise starts when the user runs the Python program. The main menu is displayed with different options for managing financial information.

1. The user starts the SpendWise program.
2. The main menu is displayed.
3. The user selects an operation such as adding income, adding an expense, viewing transactions, managing the budget, or generating a report.
4. If required, the user enters the necessary transaction details.

5. The system checks the entered values. For example, income, expense, and budget amounts must be greater than zero.
6. The valid information is stored in the appropriate Python list.
7. The selected function processes the stored information and performs the required calculations.
8. The result is displayed to the user.
9. The user can return to the main menu and perform another operation.
10. The program ends when the user selects the Exit option.

Basic Workflow

Start → Main Menu → Select Operation → Enter Data → Validate Input → Store Data → Process Data → Display Result → Main Menu / Exit

9. System Architecture

SpendWise follows a simple menu-driven architecture. The user interacts with the program through the main menu, and each menu option calls the function related to that operation.

The system can be divided into three main parts:

9.1 User Interface

The user interacts with SpendWise through the Python console. The main menu displays the available options, and the user enters a choice to perform an operation.

9.2 Processing

The different functions in the program process the user's input and perform operations such as adding transactions, calculating expenses, managing budgets, analyzing spending, calculating savings, and generating reports.

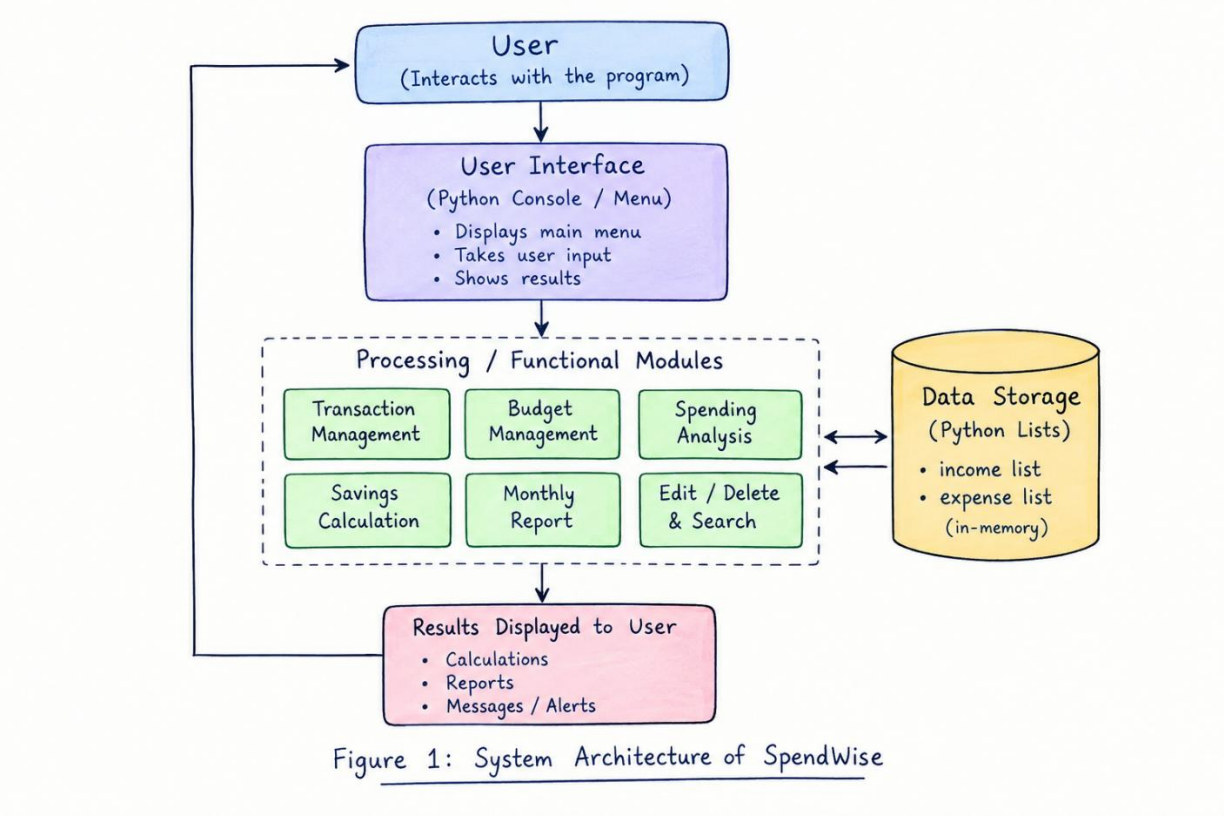
9.3 Data Storage

The current version of SpendWise uses Python lists to store income and expense transactions while the program is running. The stored information is used by different functions for calculations and displaying results.

Architecture Flow

User → Main Menu → Functional Modules → Python Lists → Processing and Calculations → Results Displayed to User

Figure 1: System Architecture of SpendWise



10. Design Decisions

While developing SpendWise, some simple design decisions were made to keep the project easy to use and easy to understand.

10.1 Menu-Driven Interface

A menu-driven interface was used so that the user can select different operations easily. The main menu provides options such as adding transactions, managing the budget, viewing savings, generating reports, and searching transactions.

10.2 Use of Functions

Different operations are written as separate functions. For example, there are separate functions for adding income, adding expenses, managing the budget, and generating reports. This makes the program easier to understand and modify.

10.3 Use of Python Lists

Python lists are used to store income and expense transactions while the program is running. Two separate lists are used for income and expenses, making it easier to process them separately.

10.4 Transaction Details

Each transaction stores important information such as transaction ID, amount, category, description, month/year, and payment method. This information is used for viewing, editing, deleting, searching, and calculating financial summaries.

10.5 Input Validation

Basic validation is included to prevent invalid amounts. Income, expenses, and budgets are not accepted when the entered amount is zero or negative.

10.6 Simple Calculations

The project uses basic calculations to find total expenses, savings, budget usage, savings percentage, and category-wise spending. This keeps the system simple while providing useful financial information.

11. Implementation Details

SpendWise was implemented using Python. The program is written as a single Python file and uses basic Python programming concepts to perform different operations.

11.1 Programming Concepts Used

The following Python concepts were used in the project:

- Variables for storing values such as amounts, categories, and budgets.
- Input and output using input() and print().
- Lists for storing income and expense transactions.
- Conditional statements such as if, elif, and else for decision-making.
- for and while loops for processing transactions and displaying the menu repeatedly.
- Functions to divide the program into different operations.
- Basic calculations for expenses, savings, budget usage, and percentages.
- List methods such as append() and pop() for adding and deleting transactions.
- String methods such as lower() for searching transactions without considering letter case.

11.2 Transaction Implementation

Income and expense details are stored in separate Python lists. Each transaction contains an ID, amount, category, description, month/year, and payment method.

When a new transaction is added, its details are stored in the appropriate list. The stored information is later used by other functions for viewing, editing, deleting, searching, and calculations.

11.3 Budget Implementation

The budget function takes the month and budget amount from the user. It calculates the total expenses for that month and compares them with the budget.

The percentage of the budget used is calculated using:

Budget Used (%) = (Total Expenses / Budget) × 100

Based on the percentage, the system displays an appropriate budget status or warning.

11.4 Spending and Savings Implementation

The spending analysis function checks the expense categories and calculates the amount spent in each category. It also calculates the percentage of total expenses for each category.

The savings function calculates savings using:

Savings = Total Income – Total Expenses

The savings percentage is calculated as:

Savings Percentage = (Savings / Total Income) × 100

11.5 Monthly Report Implementation

The monthly report function takes a month and year from the user and processes only the transactions belonging to that month. It displays total income, total expenses, savings, savings percentage, and category-wise expenses.

11.6 Edit, Delete and Search Implementation

The edit function allows the user to select a transaction using its ID and enter new details. The delete function removes the selected transaction from the appropriate list using pop().

The search function checks the category and description of transactions. The lower() method is

12. Screenshots and Results

Figure 2: Main Menu

```
========== SPENDWISE ==========
1. Add Income
2. Add Expense
3. View Transactions
4. Manage Budget
5. View Spending Analysis
6. View Savings
7. Generate Monthly Report
8. Edit Transaction
9. Delete Transaction
10. Search Transaction
11. Exit
Enter your choice:
```

Figure 3: Adding an Expense

```
Enter expense amount: 200
Enter expense category: transport
Enter description: bus and auto
Enter month and year (MM/YYYY): 9/26
Enter payment method: cash

Expense added successfully!
```

```
========== SPENDWISE ==========
1. Add Income
2. Add Expense
3. View Transactions
4. Manage Budget
5. View Spending Analysis
6. View Savings
7. Generate Monthly Report
8. Edit Transaction
9. Delete Transaction
10. Search Transaction
11. Exit
Enter your choice: 2
Enter expense amount: 500
Enter expense category: FOOD
Enter description: food and snacks
Enter month and year (MM/YYYY): 09/26
Enter payment method: UPI

Expense added successfully!
```

Figure 4: adding in income

```
1. Add Income
2. Add Expense
3. View Transactions
4. Manage Budget
5. View Spending Analysis
6. View Savings
7. Generate Monthly Report
8. Edit Transaction
9. Delete Transaction
10. Search Transaction
11. Exit
Enter your choice: 1
Enter income amount: 5000
Enter income category: pocket money
Enter description: pocket money
Enter month and year (MM/YYYY): 07/26
Enter payment method: UPI

Income added successfully!
```

Figure 4: Viewing Transactions

```
Enter your choice: 3

===== TRANSACTIONS =====

--- INCOME ---
No income recorded.

--- EXPENSES ---

Expense ID: 1
Amount: 500.0
Category: FOOD
Description: food and snacks
Month/Year: 09/26
Payment Method: UPI

Expense ID: 2
Amount: 200.0
Category: transport
Description: bus and auto
Month/Year: 9/26
Payment Method: cash
```

Figure 5: Budget Management

```
===== BUDGET MANAGEMENT =====
Enter monthly budget: 2000

Total Budget: ₹ 2000.0
Total Expenses: ₹ 700.0
Remaining Budget: ₹ 1300.0
Budget Used: 35.0 %
Status: Budget is under control.
```

Figure 6: Spending Analysis

```
========== SPENDWISE ==========
1. Add Income
2. Add Expense
3. View Transactions
4. Manage Budget
5. View Spending Analysis
6. View Savings
7. Generate Monthly Report
8. Edit Transaction
9. Delete Transaction
10. Search Transaction
11. Exit
Enter your choice: 6

===== SAVINGS =====
Total Income: 5000.0
Total Expenses: 700.0
Savings: 4300.0
Savings Percentage: 86.0 %
```

Figure 7: Monthly Report

```
========== SPENDWISE ==========
1. Add Income
2. Add Expense
3. View Transactions
4. Manage Budget
5. View Spending Analysis
6. View Savings
7. Generate Monthly Report
8. Edit Transaction
9. Delete Transaction
10. Search Transaction
11. Exit
Enter your choice: 7
Enter month and year for report (MM/YYYY): 09/26

===== MONTHLY REPORT =====
Month: 09/26
Total Income: 0
Total Expenses: 500.0
Savings: -500.0
Savings Percentage: 0 %

--- CATEGORY-WISE EXPENSES ---
FOOD: 500.0 ( 100.0 %)
```

13. Testing

Testing was performed on SpendWise to check whether the different features were working correctly and producing the expected results. Different normal and invalid inputs were used during testing.

13.1 Functional Testing

Test Case	Input/Action	Expected Result	Result
Add Income	Enter a valid income amount and details	Income should be added successfully	Passed
Add Expense	Enter a valid expense amount and details	Expense should be added successfully	Passed
View Transactions	Select View Transactions	Stored transactions should be displayed	Passed
Manage Budget	Enter a valid monthly budget	Budget usage and remaining amount should be calculated	Passed
Budget Alert	Use 75%, 90%, 100%, and more than 100% of budget	Appropriate warning should be displayed	Passed

Test Case	Input/Action	Expected Result	Result
Spending Analysis	Add expenses in different categories	Category-wise spending and percentages should be displayed	Passed
Savings	Enter income and expenses	Savings and savings percentage should be calculated	Passed
Monthly Report	Enter a valid month/year	Monthly financial details should be displayed	Passed
Edit Transaction	Enter a valid transaction ID	Transaction details should be updated	Passed
Delete Transaction	Enter a valid transaction ID	Transaction should be deleted	Passed
Search Transaction	Search using category or description	Matching transactions should be displayed	Passed

13.2 Input Validation Testing

The program was also tested with invalid values to check whether incorrect amounts were rejected.

Test	Input	Expected Result	Result
Expense Validation	Expense = 0	Amount should be rejected	Passed
Income Validation	Income = 0	Amount should be rejected	Passed
Budget Validation	Budget = 0	Budget should be rejected	Passed
Budget Exceeded	Expenses greater than budget	Budget exceeded warning should be displayed	Passed
Case-Insensitive Search	Search using lowercase text	Matching transaction should be found	Passed

13.3 Testing Result

The testing showed that the major functions of SpendWise worked correctly with the tested inputs. The program successfully handled transaction management, budget calculations, spending analysis, savings calculations, monthly reports, and basic input validation.

14. Challenges Faced

During the development of SpendWise, I faced some difficulties while writing and testing the program.

14.1 Understanding Program Logic

At the beginning, it was difficult to understand how different parts of the program should work together. I had to divide the project into smaller functions and understand the logic of each feature separately.

14.2 Handling Transactions

Managing income and expense transactions using lists required careful use of indexing and loops. It was important to access the correct transaction details while viewing, editing, deleting, and searching.

14.3 Budget Calculations

Calculating the total expenses for a particular month and finding the percentage of the budget used required careful checking of the conditions and calculations.

14.4 Debugging Errors

While developing the program, some errors occurred due to incorrect indentation, conditions, and calculations. Testing the program repeatedly helped in finding and correcting these errors.

14.5 Testing Different Inputs

It was necessary to test the program with different values, including zero amounts, different budget limits, different categories, and different search inputs. This helped in checking whether the program worked correctly in diff

15. Learnings

Working on the SpendWise project helped me understand how Python concepts can be used to create a complete working program.

15.1 Python Programming

I improved my understanding of variables, lists, loops, conditional statements, functions, and basic calculations by using them in a real project.

15.2 Use of Functions

I learned how to divide a large program into smaller functions. Each function performs a specific task, which makes the program easier to understand and manage.

15.3 Working with Lists

I learned how lists can be used to store and manage multiple pieces of information. I also learned how to add and remove data using list methods.

15.4 Problem Solving

The project helped me improve my problem-solving skills. I learned how to break a problem into smaller parts and develop each part step by step.

15.5 Debugging and Testing

I learned that testing is important for finding mistakes in a program. By testing different inputs, I was able to identify and correct errors in the program.

15.6 Project Development

I learned that developing a project requires planning, implementation, testing, documentation, and making improvements based on the result

16. Future Enhancements

The current version of SpendWise provides the main features required for personal expense tracking. In the future, the system can be improved by adding more features.

16.1 Permanent Data Storage

The system can be connected to file storage or a database so that the transactions are not lost when the program is closed.

16.2 Graphical User Interface

A graphical user interface can be added instead of the current console-based interface. This can make the application easier to use.

16.3 More Detailed Reports

Future versions can provide reports for different time periods, such as weekly, monthly, and yearly reports.

16.4 Data Visualization

Charts and graphs can be added to show spending patterns and category-wise expenses in a more visual way.

16.5 Improved Budget Management

The system can allow users to set separate budgets for different categories such as food, transport, and shopping.

16.6 User Login

A login system can be added so that different users can maintain their own financial records separately.

16.7 Mobile Application

SpendWise can be developed as a mobile application so that users can record their expenses easily while travelling or making purchases.

17. References

The following resources were referred to during the development of the SpendWise project:

1. Python Software Foundation. Python Documentation. Python Documentation
2. VIT Bhopal University. Problem Solving Using Python – Course Material.
3. VIT Bhopal University. Project Assignment Guidelines.
4. Class notes and examples provided during the Problem Solving Using Python course.